

一个共享资源（即临界区）的互斥访问，而信号量则是用于协调多个线程对一系列共享资源的有序操作。例如，互斥锁可以用于保护生产者消费者问题中的共享缓冲区，而信号量用于协调生产者与消费者何时该等待，何时该放入或拿取缓冲区数据。

条件变量与信号量提供了类似的操作接口（包括 `wait` 与 `signal`），因此很容易混淆。实际上在很多情况下（比如本节的生产者消费者问题），条件变量与信号量都能够满足同步需求。它们之间的区别是层级之间的区别。条件变量提供了一个操作系统辅助的睡眠与唤醒机制（场景二），而信号量则是针对其中一个特定的场景（场景三），提供了对有限资源管理更加方便易用的接口供开发者使用。在生产者和消费者的实现中，开发者同样可以使用“互斥锁 + 计数器 + 条件变量”实现与信号量同样的功能。但对于某些其他场景，信号量就不再方便使用，也即条件变量的适用范围更加广泛。一个比较直观的例子是，针对事务处理场景，如果处理线程等待事务到达后希望一次性拿走所有的待处理事务一并处理，使用信号量就很难实现——信号量每次执行 `wait` 操作只能拿走其中的一个资源。总而言之，条件变量能够在更广泛的场景中使用，而信号量则是管理有限资源时更方便的抽象。

练习

9.5 下面是一系列系统软件中可能会遇到的场景，请将其归纳到本章介绍的场景中，并阐释如何使用本章介绍的同步原语解决问题。

- (a) 多线程应用程序中常用到的一种工具叫作多线程执行屏障。其作用如下：假设现在有多个线程同时运行，运行到执行屏障后，线程不再继续往下运行，而是等待所有的线程都执行到该屏障之后再继续执行。如何协同这些多线程实现执行屏障的功能？
- (b) 操作系统多核调度器的每个核心都有自己的等待队列。CPU 核心将在自己的等待队列中选取合适的线程执行。现在需要一种支持工作窃取的负载均衡机制：当自己的等待队列为空时，可以去其他核心等待队列窃取任务执行，从而达到负载均衡。如何协调多个核心上的调度线程？
- (c) 并行计算程序通常使用一种映射 - 归约（Map Reduce）模式将较大的任务拆分成多个可以并行执行的小任务，然后将这些小任务的计算结果合并起来成为最终结果。比如在并行程序统计字符数的场景下，会创建多个工作线程用于拆分统计某大文件中的字符数。工作线程执行完成后由一个收集线程收集每个工作线程统计的字数并累加。工作线程完成时间不等，一旦有任何一个工作线程完成执行，收集线程就应该开始累加，而不是等待全部执行完成后再累加。如何协调工作线程与收集线程？
- (d) 某网页渲染时需要同时发多个请求，并等待全部请求都成功后再完成页面渲

染。这些请求都带有回调函数。如何在回调函数中采用同步原语完成该需求？

(e) 某服务器需要控制处理请求线程的并行数量，避免过多线程同时运行。如何协调这些线程的执行？

(f) 网页服务器中，有的线程用于响应客户端获取网页的需求，有的线程用于后端实时更新网页。如何协调这些线程对于网页数据的访问？

9.3 死锁

本节主要知识点

- ❑ 发生死锁的四个必要条件是什么？
- ❑ 如何检测一个系统中是否存在死锁？
- ❑ 在发生死锁后如何恢复到正常状态？
- ❑ 如何在源头上预防死锁？
- ❑ 如何使用银行家算法在运行时避免产生死锁？

9.3.1 死锁的定义

同步原语在给开发者带来便利的同时，也带来了一系列问题。本节将介绍其中最为经典的问题：死锁问题。下面我们通过一个简单的例子来介绍何为死锁。假设线程 A 与线程 B 分别运行代码片段 9.17 中的 `thread_A` 与 `thread_B`。这两个线程共享两把全局的互斥锁 `lock_A` 与 `lock_B`。线程 A 先尝试获取锁 A（第 5 行），再尝试获取锁 B（第 7 行），而线程 B 刚好相反。假设在 T_0 时刻，线程 A 获取了锁 A（第 5 行），将要尝试获取锁 B。而线程 B 刚好也获取了锁 B（第 15 行），等待获取锁 A。但是由于两个线程都持有对方将要获取的锁，因此这两个线程都不能进入临界区继续执行，这种互相等待的僵持状态即死锁现象。当有多个（两个及以上）线程竞争有限数量的资源时，有的线程就会因为某一时刻没有空闲的资源而陷入等待。而死锁（Deadlock）就是指这一组中的每一个线程都在等待组内其他线程释放资源而造成的无限等待。

代码片段 9.17 死锁示例

```
1 struct lock lock_A, lock_B;
2
3 void thread_A(void)
4 {
5     lock(&lock_A);
6     // T0 时刻
7     lock(&lock_B);
```

```

8   // 临界区
9   unlock(&lock_B);
10  unlock(&lock_A);
11 }
12
13 void thread_B(void)
14 {
15     lock(&lock_B);
16     // T0 时刻
17     lock(&lock_A);
18     // 临界区
19     unlock(&lock_A);
20     unlock(&lock_B);
21 }

```

产生死锁的原因

从上述例子中我们可以初步总结出：死锁是由于资源有限（如锁 A 与锁 B 都只能同时被一个线程持有）且线程的执行交错导致的。实际上，导致死锁出现一共有四个必要条件：

- 互斥访问。9.2.1 节介绍过，互斥访问保证一个共享资源在同一个时刻只能被至多一个线程访问。在有互斥访问的前提下，线程才会出现等待。
- 持有并等待。线程持有一些资源，并等待一些资源。如在代码片段 9.17 中，两个线程在持有一把互斥锁的同时等待对方放锁。
- 资源非抢占。一旦一个资源（比如例子中的锁）被持有，除非持有者主动放弃，其他竞争者都得不到这个资源。
- 循环等待。循环等待^①是指存在一系列线程 $T_0, T_1, \dots, T_n$ ，其中 T_0 等待 T_1 ， T_1 等待 T_2 ， $\dots$ ， T_{n-1} 等待 T_n ，而 T_n 等待 T_0 。因此形成了一个循环。由于循环中任意一个线程都无法等到资源，因而不能释放已经占有的资源，便出现了循环等待。

注意，死锁虽然带了锁字，但不意味着只有在使用互斥锁时才会出现。只要满足上面四个必要条件，该系统中就会出现死锁。

除了多个线程相互阻塞会造成死锁之外，还有一种特殊的情况也会导致死锁：如在中断处理流程中使用了互斥锁，那么一旦在某一中断处理流程中获取锁后、放锁前再次出现中断，就会导致死锁。此外，在递归函数中使用互斥锁也会面临同样的问题。如代码片段 9.18 所示，如果线程需要在递归函数 `funcA_recursive` 中获取互斥锁 `global_lock`；其有可能在持有互斥锁时再次调用 `funcA_recursive`（第 7 行），造成此次调用无法再获取互斥锁 `global_lock`（第 5 行），因此发生了死锁。

① 注意这里的循环等待与前文的循环等待（Busy Looping）不是同一个概念。前文的循环等待是指利用 `while` 来不断重复判断指定条件，等待该条件得到满足。

代码片段 9.18 递归产生的死锁

```
1 struct lock global_lock;
2
3 int funcA_recursive(int arg)
4 {
5     lock(&global_lock);
6     if (some_condition)
7         funcA_recursive(some_arg);
8     unlock(&global_lock);
9     // .....
10 }
```

不过，这类死锁可以通过使用可重入锁来解决。可重入锁在加锁时会判断锁的持有者是不是线程自己，如果是自己则不会阻塞并等待，而是通过一个计数器来记录加锁次数。这个计数器会在放锁时减少，当为 0 时才真正放锁。此外，在操作系统中断处理流程中，这个问题往往通过在加锁时关中断、放锁时开中断来解决。这样能够避免在临界区内触发另外的中断处理流程导致锁的嵌套造成的死锁。后续小节将不再讨论此类死锁。

9.3.2 死锁检测与恢复

当死锁出现时，我们需要一个第三者来打破僵局，帮助死锁恢复。操作系统往往扮演了这个第三者的角色。由于死锁时线程进入了无限的等待状态，无法完成有价值的工作，因此操作系统可以判断死锁是否发生，并尽量使用最小的代价恢复系统的正常运行。

结合之前介绍的死锁出现的四个必要条件，我们来分析如何检测系统中是否出现了死锁。由于四个必要条件中的前三个都是描述使用场景本身的性质，只有最后一条（即循环等待）和实际运行状态相关，因此循环等待是检测死锁的关键。

为了确认系统中是否存在循环等待，需要获取系统中资源分配与线程等待的相关信息。系统通过两个表（资源分配表和线程等待表）来记录这些数据。如图 9.8 所示，假设现在有四个线程 $T_1 \sim T_4$ ，而它们之间共享三个资源 $O_1 \sim O_3$ 。资源分配表记录当前不同资源被占有的情况，而线程等待表记录线程等待不同资源的情况。根据这两张表，可以画出图 9.8 左边的图。其中从资源指向线程的实线箭头代表线程占有该资源，而从线程指向资源的虚线箭头代表线程等待该资源。对于其中的任意一个资源，指向其的虚线箭头与从其指出的实线箭头都构建成了一对线程间的等待关系。如对于资源 O_1 ，它有从 T_4 与 T_2 指向其的虚线箭头，也有指向 T_1 的实线箭头，因此 T_4 与 T_1 、 T_2 与 T_1 均构成了等待关系。所以，在该图中如果出现环，就代表出现了循环等待。在图 9.8 的例子中，线程子集 T_1 、 T_3 、 T_4 之间就形成这样的环，出现了循环等待，因此我们可以判断此时出现了死锁。

如果通过分析资源分配 / 等待图发现了死锁，下一步就需要从死锁中恢复。为了让系统恢复正常运行，我们需要打破循环等待的关系。最直接的方法是，找到这个环中任

意的线程作为受害者，直接终止该线程并释放其占有的资源。如果由于分配 / 等待关系过于复杂，在终止线程后依旧成环，就继续选择下一个线程并释放其占有的资源。通过一步步终止环中的线程，可以打破循环等待，恢复正常运行。除了这种不太公平的策略外，还可以让环上所有线程回退到之前的某一个状态再次运行。由于死锁出现往往是由于特定的调度和触发时机导致，再次运行有很大概率不会触发死锁。

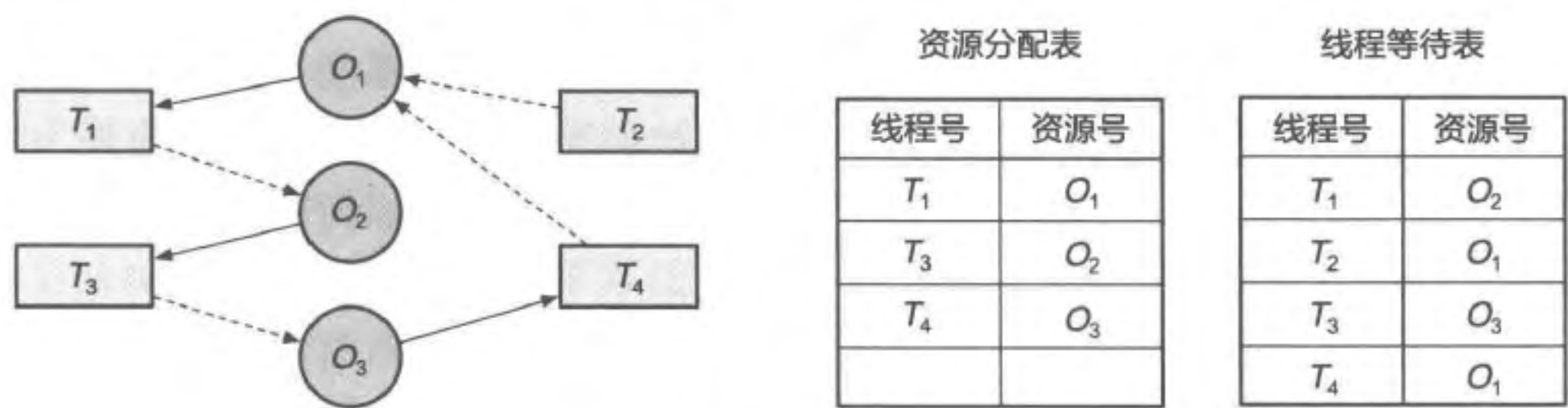


图 9.8 死锁检测

那么，操作系统到底应该何时开始做死锁检测呢？从我们介绍的算法可以发现，检测死锁并不是一件轻松的工作，操作系统需要构建一个十分复杂的图并在其中寻找环。因此频繁的死锁检测会造成严重的性能开销。操作系统往往选用更加被动的策略触发死锁检测，避免暴露巨大的开销在关键路径上以减少对正常执行时的应用程序性能的影响。通常选用的策略有定时检测（如运行 24 小时检测一次）或超时等待检测（如等待某资源超过限定的时间时检测一次）。

除了“事后诸葛亮”式的在出现死锁后再恢复以外，研究者还提出了一系列方法直接避免死锁的出现。这些方法主要分为两类：死锁预防与死锁避免。其中，死锁预防是指从源头设计上避免死锁的出现，而死锁避免则是在运行时动态检查以避免死锁的出现。接下来我们将分别介绍死锁预防与死锁避免的相关技术。

9.3.3 死锁预防

死锁预防是指通过设计合理的资源分配算法，从源头上预防死锁。为了实现这个目标，只需要保证产生死锁的四个必要条件不被同时满足。下面我们根据之前列举的死锁产生的条件，一条条看怎么从源头设计上就避免这些条件成立。

- 避免互斥访问。互斥访问是用于保护共享资源的。如果需要避免互斥访问，就需要设计其他的方法来保证同时访问共享资源的正确性。比如设计一个代理线程，专门用于管理对于共享数据的访问与修改。这个共享数据只能通过代理线程来操作，其他线程需要向代理线程发送请求来完成对共享数据的访问。如在刚才产生死锁的例子中（代码片段 9.17），我们可以将 thread_A 与 thread_B 的临界区都放在一个代理线程中执行。加锁与放锁的流程分别改为向代理服务器发送请求与接收代理服务器执行结果。这样就可以避免由于不同的线程占有并等待一部分

资源导致死锁（所有资源只会由一个线程获取）。这种设计的主要问题是大部分应用程序不容易修改成此模式，而且对于每一个共享资源，都需要多余的一个代理线程来收集并执行对其的修改，这将为系统带来很大的负担。

- 不允许持有并等待。为了打破“持有并等待”这一条件，可以要求线程在真正开始操作之前一次性申请所有的资源。一旦需要获取的资源中任一资源不可用，则该线程就不能成功申请这一系列资源，且该线程必须释放已经占有的资源。这种方法的问题在于，当资源的竞争程度高时，线程很可能不能一次性拿到所有共享资源的使用权。因此可能会进入申请-释放的循环，造成资源利用率低，甚至出现饥饿（Starvation）的情况。我们将在 9.4 节详细介绍如何将刚才的例子转换为这种模式，以及其带来的新问题。
- 允许资源被抢占。这将允许一个线程抢占其他线程已经占有的资源，但难点是需要保证被抢占的线程能够正确恢复。比如在刚才的例子中，如果允许线程 B 抢占线程 A 已经持有的锁 A，那么就需要回滚线程 A 在持有锁 A 之后进行的操作，然后允许线程 B 获取锁 A。待线程 B 释放锁 A 后，再恢复线程 A 之前的修改。显然，在这个例子中，回滚与恢复操作十分困难，因此这种策略也只适用于易于保存和恢复的场景。
- 避免循环等待。为了避免出现循环等待，我们可以要求线程必须按照一定顺序来获取资源。通过这种方法，获取资源进度最靠前的线程一定能够拿到所有所需的资源，从而避免出现循环等待。这种资源有序分配算法会将系统中的所有资源进行编号，线程在申请资源时，必须按照资源编号递增来申请。考虑在任一时刻，系统中总会有一个得到资源编号最大资源的线程，这也意味着这个线程之后需要申请的资源都还没有被人占有，因此该线程可以继续执行。而正是因为该系统在任意时间点都符合上面的描述，所以这个系统不会陷入因循环等待而无法继续执行的状态。在之前产生死锁的例子中，我们可以对锁 A 与锁 B 进行编号，要求所有线程按照顺序获取锁 A 与锁 B。这样就可以避免这两个线程发生死锁。

9.3.4 死锁避免

死锁避免在系统运行时跟踪资源分配过程来避免出现死锁。在系统运行时，任意线程在需要新的资源时都必须向系统提出申请。而系统将根据其所处状态，判断是否能够将资源分配给该线程。系统存在以下两种状态：安全状态与非安全状态。安全状态是指系统中存在至少一个安全序列 $\{T_1, T_2, \dots, T_n\}$ 。如果系统按照这个序列 $(T_1, T_2, \dots)$ 调度线程执行，即可避免资源不足的情况发生。可能有读者对“至少一个”的要求感到疑惑，如何保证系统在运行时一定能按照这个序列进行呢？这一点就是死锁避免的特点所在，在这种系统中，每一次线程需要获取资源时，都由系统选择直接分配还是让其等待。因此如果系统至少有一个安全序列，那么系统就会在线程申请资源时找到这个序列，

并按照这个序列继续推进。而如果系统中不存在这样一个安全序列，则称之为非安全状态。需要注意的是，非安全状态不一定导致死锁。非安全状态是指，在线程必须获取所有所需资源后才能结束且在此前不会释放已经占有的资源的前提下，一定会导致死锁。否则，非安全状态只是指系统按照静态给出的需求无法找到安全序列，而实际上应用程序可能会在运行中途释放一些资源或者不需要其声明的所有资源就能运行结束。此时也不会导致死锁。总而言之，安全状态一定不会出现死锁，而非安全状态可能会导致死锁。而死锁避免算法就是让系统在每一次分配资源后都处于安全状态。如果分配之后不能处于安全状态，则不分配资源给线程。

下面，我们将详细介绍一种具体的死锁避免算法——银行家算法，它同样是由 Edsger Dijkstra 提出的^[8]。银行家算法通过模拟分配资源后的状态来判断在分配某个资源后，系统是否还处于安全状态。为了描述系统中的供给/需求关系，银行家算法使用了一系列数据结构来保存这些关系。假设我们的系统中存在 M 类资源，而线程有 N 个，则这些数据结构为：

- 全局可利用资源：Available[M]。该数组代表某一时刻系统中每一类元素的可用个数。这个数组初始化时设置为系统中拥有的 M 类资源的总量。
- 每个线程的最大需求量：Max[N][M]。该矩阵包含所有 N 个线程对 M 类资源的最大需求量。如线程 i 对资源 j 的最大需求量为 Max[i][j]。
- 已分配资源数量：Allocation[N][M]。该矩阵包含已经分配给 N 个线程中每个线程的 M 类资源的数量。
- 还需要的资源数量：Need[N][M]。该矩阵包含所有 N 个线程对 M 类资源还需要的资源数量。

此外，我们还需要保证以下前提，银行家算法才能正确工作：

- 系统中的供给关系必须是固定的。比如系统中的资源类别数 M ，线程总数 N ，整个系统中 M 类资源的可用数量，以及每个线程所需要的不同种类资源的数量。
- 任意线程对于任意资源的最大需求量不能超过系统的资源总量。
- 对于任意一类资源，任意线程已分配的资源加上该线程还需要的资源，必须要小于等于该线程对于这一类资源的最大需求量。
- 线程在获得了所有需要的资源之后，能够在有限时间内完成工作，并且释放已经获得的资源。

对于满足这些前提的系统，我们可以利用银行家算法保证该系统一直处于安全状态。正如之前所述，银行家算法的核心是模拟分配某个资源之后，检查系统是否还处于安全状态。如果发现分配了该资源之后，系统无法处于安全状态，则不会分配该资源，从而保证系统能够始终处于安全状态。这个检查算法被称为安全性检查算法。安全性检查算法的执行流程如下：

1. 创建一个临时数组 Available_sim，其初始值与数组 Available 一致，用于

记录在接下来的模拟执行中系统可用的资源数量。

2. 找到当前系统剩余资源能够满足的线程。即找到一个线程 x ，对于 $Available_sim$ 中任意成员 m ，都满足 $Available[m] \geq Need[x][m]$ 。如果无法找到这样的线程，则代表系统处于非安全状态。

3. 假设将资源全部分配给它，且它执行完成后会释放所有的资源。即对于 $Available_sim$ 中的所有成员 m ，都执行 $Available_sim[m] += Allocation[x][m]$ 。同时标记线程 x 执行结束。

4. 遍历所有线程，查看是否还有未被标记执行结束的线程。如果有则回到第二步继续执行，否则代表系统处于安全状态。

下面我们结合一个具体例子来详细描述安全性检查算法的具体执行过程。

假设系统中一共有 T_1 、 T_2 、 T_3 三个线程，它们均需要 A 、 B 这两类资源。在某个时刻，银行家算法中各个数据结构的状态如表 9.2 所示。此时我们执行安全性检查算法，检查该时刻系统是否安全。安全性检查算法首先找到当前可以满足需求的线程，并假设将资源全部分配给该线程。在这个例子中，我们发现该时刻可以满足需求的线程为 T_2 。因此假设满足 T_2 的需求， T_2 应当拿到资源在有限时间内执行完成，然后释放自己已经拥有的资源。

表 9.2 银行家算法示例：初始状态

	Max		Allocation		Need		Available	
	A	B	A	B	A	B	A	B
T_1	5	10	2	8	3	2	3	1
T_2	3	1	0	1	3	0		
T_3	10	11	5	1	5	10		

此时系统的状态应当如表 9.3 所示。对比表 9.2 可以发现，除了我们将 T_2 的需求清除之外， T_2 还释放了自己的资源，并且将该资源放到 $Available$ 中。而此时 T_1 的请求可以得到满足。同理，我们进一步假设将资源分配给 T_1 ，并释放其已经分配的资源。在 T_1 执行结束后， A 与 B 全局可用的资源分别为 $3 + 2 = 5$ 与 $8 + 2 = 10$ 。而这刚好可以满足 T_3 的需求，使得 T_3 能够顺利执行。因此，我们可以得到一个安全序列 $\{T_2, T_1, T_3\}$ ，该序列能够使系统顺利运行完所有的线程，其表明该系统处于安全状态。

表 9.3 银行家算法示例：假设 T_2 执行完成

	Max		Allocation		Need		Available_sim	
	A	B	A	B	A	B	A	B
T_1	5	10	2	8	3	2	3	2
T_2								
T_3	10	11	5	1	5	10		

银行家算法就是在每次分配给线程新的资源时都进行安全性检查，保证这个分配不会导致系统进入非安全状态。如在表 9.2 所示状态中，假设此时 T_1 发出请求申请资源 A ，申请数量为 1，虽然剩余的资源数量大于这个请求，但是若将 A 资源分配给 T_1 ，系统将无法找到一个可以满足的线程，因此处于非安全状态。此时，系统会安排 T_1 等待，直到其他线程（如 T_2 ）顺利获取资源并释放占有的资源时才再次进行安全性检查。只有在安全性检查通过之后，线程的请求才能够得到满足。

回到本小节开头产生死锁的例子中，我们该如何使用死锁避免算法来消除死锁？这里，我们可以把代码片段 9.17 中的锁 A 与锁 B 都当成仅有 1 份的资源。而线程 A 与线程 B 都需要获取锁 A 与锁 B 。如果线程 A 先获取其中的锁 A ，此时线程 B 向系统申请锁 B ，系统就会检查如果把锁 B 分配给线程 B ，是否会导致系统处于非安全状态。显然，如果此时分配给线程 B ，那么系统将无法找到安全序列。因而系统不会立刻将锁 B 分配给线程 B ，而是待线程 A 结束执行后才分配，从而避免了死锁的产生。

9.3.5 哲学家问题

哲学家问题是一个经典的死锁案例。哲学家问题描述了一个非常有意思的场景：如图 9.9 所示，现在有 5 名哲学家 A、B、C、D、E 围着一个圆形餐桌落座。每一位哲学家面前放着一份食物，左手与右手边分别放着一支筷子。每一位哲学家必须同时拿起这两支筷子才能开始进食。然而，筷子数量是有限的：相邻的哲学家中间只有一支筷子。当哲学家感到饥饿时，他将尝试拿起这两支筷子。如果成功获取，则开始进食，并在吃饱后放下筷子；否则将一直拿着已经获取的筷子并等待邻座的哲学家放下

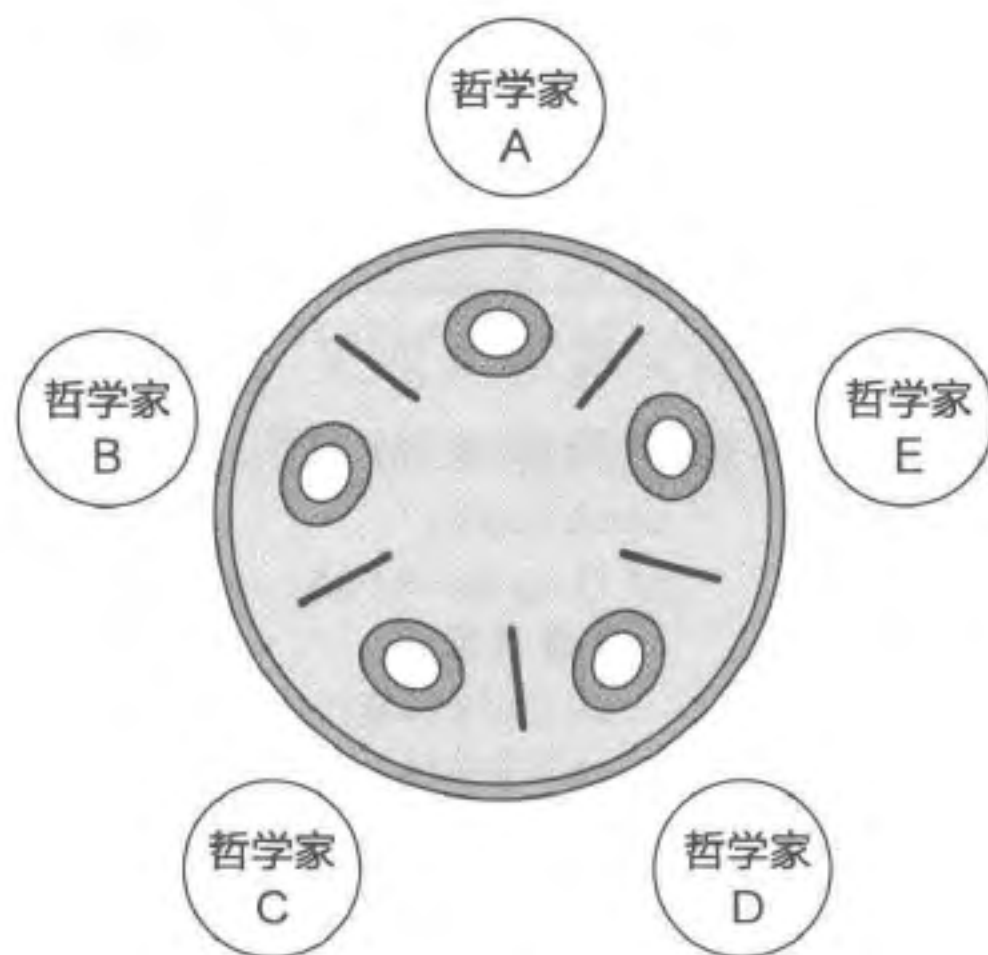


图 9.9 哲学家问题

筷子。现在假设食物是无限的，而哲学家在思考问题的间隙如果感到饥饿就会尝试拿起筷子进食。

在理想的情况下，哲学家可以依次使用筷子，从而都可以吃到食物。然而，存在这样的一种情况：当所有的哲学家都尝试进食且都拿起了他们左手边的筷子时，每一位哲学家都在等待右手边的哲学家放下筷子（如哲学家 A 需要等待哲学家 B 放下筷子）。由于每一位哲学家都不肯让步，没有人会放下已经拿到的筷子，因此这些哲学家永远无法拿到右手边的筷子。此时，我们可以认为发生了死锁：一支筷子只能被一个哲学家使用（互斥访问），哲学家占有了一支筷子并等待另一支（持有并等待），哲学家无法从别人手上抢筷子（资源非抢占），并且形成了哲学家 A 到 B、B 到 C、C 到 D、D 到 E、E 又到 A 的循环等待。

9.4 活锁

本节主要知识点

- 活锁与死锁的区别有哪些？
- 如何解决活锁问题？

在介绍死锁预防时，我们提到了一种不允许持有并等待的方法。这种方法能够有效避免死锁，但是会带来一个新的问题：活锁（Livelock）。代码片段 9.19 展示了使用这种策略的具体代码。线程 A 在成功获取锁 A 之后（第 4 行），不同于直接使用阻塞上锁操作拿取锁 B，这里使用 trylock 进行尝试（第 5 行）。trylock 如果成功获取锁，则返回 SUCC，否则返回 FAIL。为了避免持有并等待，一旦获取锁 B 失败，该线程将立刻放弃锁 A（第 11 行），并重新尝试获取锁 A 与锁 B。线程 B 与线程 A 同理。在 9.3.1 节产生死锁的例子中，如果采用如上策略，线程 A 在发现锁 B 上锁之后，会释放锁 A 并重试。因而线程 B 有机会拿到锁 A 并打破死锁。

代码片段 9.19 通过避免持有并等待来避免死锁

```
1 void thread_A(void)
2 {
3     while(TRUE) {
4         lock(&A);
5         if(trylock(&B) == SUCC) {
6             // 临界区
7             unlock(&B);
8             unlock(&A);
9             vbreak;    // 退出循环
10    }
11    unlock(&A);
12 }
13 }
14
15 void thread_B(void)
16 {
17     while(TRUE) {
18         lock(&B);
19         if(trylock(&A) == SUCC) {
20             // 临界区
21             unlock(&A);
22             unlock(&B);
23             break;    // 退出循环
24         }
25         unlock(&B);
26     }
27 }
```